



SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

(Code 21CSC303J)

B.Tech (CSE-AIML) – 3rd year/6th Semester/B

Name: Adarsh Kumar Rai

Registration No: RA2211026030086



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FACULTY OF ENGINEERING & TECHNOLOGY

SRM INSTITUTE OF SCIENCE & TECHNOLOGY,

**DELHI NCR CAMPUS,
MODINAGAR**

SIKRI KALAN, DELHI MEERUT ROAD, DIST. – GHAZIABAD - 201204

<https://www.srmup.in/>

Even Semester (2024-2025)

BONAFIDE CERTIFICATE

Registration no RA2211026030086 Certified to be the **Bonafide** record of work done by **Adarsh Kumar Rai** of **6th** semester **3rd** year B. TECH degree course in SRM INSTITUTE OF SCIENCE & TECHNOLOGY, DELHI-NCR Campus for the Department of Computer Science & Engineering, in **SOFTWARE ENGINEERING AND PROJECT MANAGEMENT (Code 21CSC303J)** during the academic year 2024-25.

Ms. Juhi Singh
(Assistant Professor)

Dr. Avneesh Vashishth
Head of the Department (CSE)

Submitted for university examination held on ___/___/___ at SRM IST, NCR Campus.

Internal Examiner-I

Internal Examiner-II

Index

S.No.	Elab- Programs	Date of Experiment	Date of Submission	Signature
1	Identify the Software Project, Create Business Case, Arrive at a Problem Statement			
2	Analyze Stakeholder and User Description and Identify the appropriate Process Model			
3	Identify the Requirements, System Requirements, Functional Requirements, Non-Functional Requirements and develop a SRS Document			
4	Prepare Project Plan based on scope, Find Job roles and responsibilities, Calculate Project effort based on resources			
5	Prepare the Work, Breakdown Structure based on timelines, Risk Identification and Plan			
6	Designing a System Architecture, Use Case Diagram, ER Diagram (Database)			
7	DFD Diagram (process) (Up to Level 1), Class Diagram (Applied For OOPS based Project)			
8	Create Interaction Diagrams, State chart and Activity Diagrams			
9	Design State and Sequence Diagram, Deployment Diagram			
10	Sample Frontend Design (UI/UX)			
11	Sample code implementation			
12	Create a Master Test Plan, Test Case Design			
13	Manual Testing			
14	User Manual, Analysis of Costing, Effort and Resource			
15	Project based on Software Engineering			

Experiment No. 1

Aim: Identify the Software Project, Create Business Case, Arrive at a Problem Statement

Project Overview:

The Heart Disease Prediction System is a cutting-edge, AI-powered web application aimed at predicting the likelihood of a user suffering from heart disease. The prediction is based on various input features such as age, gender, chest pain type, cholesterol level, resting blood pressure, and more, derived from publicly available health datasets. The tool is developed using Python's Streamlit framework and scikit-learn for implementing machine learning models like Logistic Regression and Random Forest.

The system is designed to improve accessibility to early diagnostic tools, particularly in remote and underserved areas where medical infrastructure is lacking. Its intuitive user interface ensures that even non-technical users can input their data and receive an accurate risk assessment in real-time. This empowers users to take proactive health measures and seek professional help early, thereby reducing the risk of severe complications or mortality.

Business Case:

Cardiovascular diseases account for nearly 18 million deaths

globally each year, making early intervention critical. However, healthcare systems, especially in developing regions, are overwhelmed and under-resourced. There is a strong need for intelligent, cost-effective, and scalable solutions that can pre-screen individuals for heart-related ailments before they reach advanced stages.

This project aims to bridge that gap by offering a machine learning–driven system that can be accessed through a browser on any device, eliminating the need for dedicated diagnostic equipment. The business value includes reduced healthcare costs, earlier treatment opportunities, increased health awareness, and potential integration with telemedicine platforms.

Key Benefits:

- Predictive analytics for early heart disease detection
- Real-time, browser-based risk assessment
- Supports offline data processing for enhanced privacy
- Scalable for integration with clinics, wearable health apps, or hospitals
- Educational value for medical students or healthcare trainees

Target Users:

- Health-conscious individuals
- General physicians and cardiologists

- Rural clinics and telehealth service providers
- Researchers in predictive medicine

Problem Statement:

Current heart disease screening methods are limited in availability and often rely on physical tests. There is a need for a fast, user-friendly, and data-driven tool that can help predict the risk of heart disease using basic health parameters. Our solution aims to bridge this gap by offering an ML-powered application accessible via web browsers.

Experiment - 2

Stakeholder Analysis (Detailed)

1. Primary Users:

- **Patients:** Likely using the system for health monitoring, diagnosis support, or wellness tracking. They value **privacy, clear interfaces, and trustworthy outputs**.
- **Healthcare Professionals:** Require tools that enhance decision-making, offer explainable AI insights, and integrate into clinical workflows. Accuracy and reliability are critical.

2. Project Owners:

- **Developers & Software Engineers:** Responsible for implementing the core ML system and its features. Need a flexible model that supports iteration and frequent updates.

3. Secondary Stakeholders:

- **Clinics & Public Health Departments:** Interested in aggregated insights, patient outcomes, and data-driven decision-making.
 - **Researchers:** Use the platform for data exploration and model validation, needing access to exportable, explainable results.
-

User Characteristics (Expanded)

- **Technical Literacy:** Ranges from basic (patients) to moderate (health professionals); therefore, the interface must be **intuitive** with minimal complexity.
 - **Privacy Concerns:** Strong requirement for **data security**, compliance with standards like HIPAA/GDPR.
 - **Trust & Simplicity:** Users demand **accuracy** in results, and interfaces that convey confidence without overwhelming them.
-

Appropriate Process Model: Incremental Development Model (Agile-Aligned)

The **Incremental Development Model** is best suited for this scenario due to the following reasons:

Justification:

1. Early Delivery of Core ML Features:

- The first increment can focus on developing and deploying the **basic machine learning model**, allowing stakeholders to test its predictions early.
- Immediate feedback can be gathered from healthcare professionals on **model performance**, interpretability, and usability.

2. Iterative Improvement:

- Agile-compatible increments allow for **continuous user involvement**, enabling developers to refine the application based on **real-world feedback**.
- Example: After deploying basic diagnostic outputs, future increments can add **confidence intervals**, **explanation modules**, or **treatment suggestions**.

3. Phased Addition of Features:

- **Visualization tools** (charts, trend lines, dashboards) can be introduced in later stages, ensuring that initial feedback doesn't get diluted by too many features.
- **Export modules** (e.g., PDF reports, CSV for researchers) can be tailored after understanding the most used formats and user preferences.
- **Explainable AI (XAI) features**, such as SHAP plots or decision trees, can be integrated in response to clinician needs for model transparency.

4. Risk Management:

- Incremental development reduces the **risk of large-scale failures** by ensuring that each phase is validated before moving to the next.
- Especially valuable in healthcare, where incorrect predictions can have serious consequences.

5. Adaptability to Regulation:

- New privacy or compliance requirements (e.g., from HIPAA updates or local health laws) can be **easily incorporated in future iterations** without redoing the entire system.

Experiment - 3

Software Requirements Specification (SRS) for Virtual Diary

1. User Registration and Login

- **Purpose:** To ensure each user has a secure and personalized experience.
- **Details:**
 - Users (patients or doctors) can register using an email and password.
 - Login credentials are securely verified.
 - Optional: Role-based login (e.g., doctor vs. patient).
 - Session management ensures data privacy.

2. Input of Health Parameters

- **Purpose:** Collect medically relevant data to feed into the prediction model.
- **Input Fields:**
 - Age, sex, resting blood pressure

- Cholesterol level
- Chest pain type (categorical)
- Fasting blood sugar, resting ECG
- Maximum heart rate, exercise-induced angina, etc.
- Input validation ensures no incorrect or out-of-range data is submitted.

3. ML-based Heart Disease Prediction

- **Purpose:** Core function that uses a machine learning model to predict the likelihood of heart disease.
- **How it works:**
 - Collected health data is preprocessed and passed to a trained ML model (e.g., logistic regression, random forest).
 - The model outputs a binary prediction (disease/no disease) along with a **confidence/risk score**.

4. Result Visualization

- **Purpose:** To present predictions in a user-friendly and understandable format.
- **Types of Visuals:**
 - Bar charts showing comparison with normal ranges
 - Risk score displayed as a percentage or gauge
 - Highlighting of high-risk factors based on the input

5. Report Generation (PDF Export)

- **Purpose:** To allow users (especially doctors) to keep a record or share results.
- **Contents of the report:**
 - Input data summary
 - Prediction outcome and confidence score
 - Timestamp, user ID, and disclaimer
 - Optional: simple medical recommendations (non-prescriptive)

Non-Functional Requirements (How the system performs)

1. Prediction Accuracy $\geq 85\%$

- The ML model must be validated to ensure it achieves at least 85% accuracy (or similar performance metrics like F1-score or AUC).

- Performance should be benchmarked using clean, balanced, and representative datasets (e.g., UCI Heart Disease dataset).

2. Data Privacy Compliance

- **No external APIs or cloud-based ML predictions.**
- All data handling and model inference must occur locally to ensure compliance with health data protection standards (e.g., HIPAA, GDPR).
- Data should be stored only if necessary and encrypted in SQLite.

3. User-Friendly Interface

- Designed using **Streamlit** for minimal setup and a clean interface.
- Users should be able to run the system without needing to install multiple dependencies.
- Clear instructions, tooltips, and meaningful error messages included.



System Requirements (Technical stack and tools)

1. Python 3.x

- Core programming language.
- Preferred version: Python 3.8 or newer for compatibility with modern libraries.

2. Streamlit

- Used for building the web-based UI.
- Allows rapid prototyping and deployment without needing front-end development.
- Supports widgets for input (sliders, dropdowns, etc.) and real-time chart rendering.

3. scikit-learn

- For training and using the ML model.
- Can be used to implement:
 - Logistic Regression
 - Random Forest
 - SVM or any classifier suited to medical prediction
- Pipelines and joblib for saving/loading models.

4. SQLite

- Lightweight, serverless database to store:
 - User credentials
 - Input history (if required)
 - Prediction logs (optional, with privacy consent)

✅ Summary Table

Component	Description
User Auth	Secure login with optional role distinction
Health Input	Collects relevant cardiovascular health indicators
ML Prediction	Uses scikit-learn model to predict heart disease risk
Result Visualization	Graphs, scores, and risk factors shown in a clean layout
Report Generation	Exportable PDF with results and insights
Accuracy	Target $\geq 85\%$ with model validation and tuning
Privacy	Full local processing, no cloud interaction
Interface	Built in Streamlit, fast and simple
Tech Stack	Python 3.x, Streamlit, scikit-learn, SQLite

Experiment – 4

Core Deliverables (Key Outputs of the Project)

1. Secure Login System

- **Functionality:**
 - User registration, authentication, and secure login.
 - Password hashing (e.g., using bcrypt) for security.
 - Session management to keep users logged in safely.
- **Rationale:** Protects sensitive medical data and provides user-specific experiences.

2. Input Form for Patient Parameters

- **Functionality:**
 - Collects patient health indicators like age, cholesterol, chest pain type, heart rate, etc.
 - Input validation (e.g., age must be between 20-100).
 - Friendly UI with dropdowns, sliders, and tooltips explaining each metric.
- **Rationale:** Accurate input is essential for reliable prediction.

3. Trained ML Model Integration

- **Functionality:**
 - Pretrained ML model (e.g., Random Forest or Logistic Regression) loaded using joblib or pickle.
 - Accepts input, performs preprocessing, and returns prediction (disease / no disease + risk score).
- **Rationale:** Core intelligence of the system enabling heart disease risk assessment.

4. Prediction Display with Health Metrics

- **Functionality:**
 - Visualizes results using:
 - Risk percentage (e.g., 72% chance of heart disease)
 - Bar or pie charts comparing patient metrics to normal ranges
 - Highlighting of risky parameters (e.g., high cholesterol)
- **Rationale:** Enhances user understanding and trust in the prediction.

5. Export Feature (PDF Generation)

- **Functionality:**

- Generates downloadable PDF reports including:
 - Input summary
 - Prediction result and explanation
 - Timestamp and optional doctor comments
 - Libraries: fpdf, reportlab, or pdfkit.
 - **Rationale:** Useful for patients, doctors, or for medical record keeping.
-

Job Roles (Team Responsibilities)

1. Project Manager

- **Responsibilities:**
 - Define milestones, assign tasks, manage timelines.
 - Communicate with stakeholders.
 - Ensure deliverables are met within budget and time.
- **Tools:** Trello, JIRA, or Notion for tracking.

2. Backend Developer

- **Responsibilities:**
 - Handle user authentication logic.
 - Manage data flow, form processing, and model integration.
 - Set up SQLite for local data storage.
- **Tools:** Python, SQLite, sqlalchemy, security libraries (bcrypt, hashlib).

3. Frontend Developer (Streamlit Developer)

- **Responsibilities:**
 - Design the Streamlit UI.
 - Build forms, display results, implement navigation.
 - Add responsive layout and accessibility features.
- **Tools:** Streamlit, Python UI libraries (plotly, matplotlib,

streamlit-pdf-download plugin).

4. ML Engineer

- **Responsibilities:**

- Data cleaning, feature engineering, and model training.
- Perform model evaluation (cross-validation, confusion matrix).
- Tune hyperparameters and export the best model.

- **Tools:** scikit-learn, pandas, matplotlib, joblib.

5. QA Tester

- **Responsibilities:**

- Test user flows, prediction correctness, and UI responsiveness.
- Validate against edge cases and malformed inputs.
- Perform performance testing and accessibility checks.

- **Tools:** Manual test cases, Streamlit testing, pytest for backend logic.

Effort Estimation (Time & Duration)

Total Estimated Time: ~300 hours

Role	Estimated Hours
Project Manager	30
Backend Developer	70
Frontend Developer	60
ML Engineer	80
QA Tester	60
Total	300 hours

Development Timeline: ~5 Weeks

- **Week 1: Planning & Setup**
 - Requirements finalization
 - Initial Streamlit structure
 - Dataset exploration & model planning
 - **Week 2: Backend & ML Model**
 - Login/authentication setup
 - ML model development and first version integration
 - **Week 3: UI & Visualization**
 - Build and test patient input form
 - Integrate and test prediction output display
 - Start on visualization modules
 - **Week 4: Report Export & Optimization**
 - Implement PDF export
 - Fine-tune model and visuals
 - Perform internal testing
 - **Week 5: QA & Deployment**
 - Full system testing and bug fixes
 - Final UI polish
 - Deliver demo and documentation
-

Experiment – 5

✅ **Work Breakdown Structure (WBS)**

The WBS breaks the project into manageable phases and sub-tasks, ensuring every team member understands their responsibilities and timelines.

1. Planning and Setup

- Define project scope and success metrics (e.g., $\geq 85\%$ prediction accuracy)
 - Identify necessary tools: Python, Streamlit, scikit-learn, SQLite
 - Collect and analyze heart disease datasets (e.g., UCI dataset)
 - Set up version control (Git), task board (Trello/JIRA), and documentation base (Notion)
-

2. Model Training and Evaluation

- Data preprocessing:
 - Handle missing values, encode categorical variables, scale features
 - Model selection and training:
 - Try multiple models: Logistic Regression, Random Forest, Gradient Boosting
 - Model evaluation:
 - Use cross-validation, confusion matrix, ROC-AUC to validate
 - Save final model using joblib or pickle for later integration
-

3. UI/UX Design

- Design patient data input forms (e.g., sliders for age, dropdown for chest pain type)
- Display prediction results with:

- Risk score (%) and simple diagnosis label
 - Visual comparisons (bar charts, color-coded warnings)
 - Streamlit layout:
 - Use columns, expanders, and tooltips for clarity
 - Add accessibility features (color contrast, readable fonts)
-

4. Backend Logic Implementation

- Implement secure login system:
 - User registration, authentication, password hashing (e.g., bcrypt)
 - SQLite database setup:
 - Store login credentials, optional prediction logs
 - Data validation:
 - Prevent empty, out-of-range, or incorrectly formatted inputs
-

5. ML Integration and Testing

- Connect trained model with backend logic
 - Preprocess user input to match training format
 - Test multiple cases for prediction correctness
 - Add logic to highlight abnormal inputs (e.g., cholesterol > 250)
-

6. Report and Export Functions

- Generate downloadable PDFs:
 - Include input summary, risk prediction, timestamp, and basic interpretation
 - PDF library usage:
 - Use fpdf, reportlab, or pdfkit to generate clean reports
 - Allow re-download of previous reports (optional)
-

7. Final Testing and Documentation

- QA testing:
 - Manual and automated testing for all features
 - Test different user roles, devices, and edge cases
- Documentation:
 - User guide (how to use the app)
 - Developer guide (setup, code structure, model info)
 - Model evaluation report with metrics and justifications

⚠ Risks and Management Strategies

Risk ID	Risk Description	Mitigation Strategy
R1	Model Inaccuracy	Use ensemble models (e.g., Random Forest, XGBoost); apply k-fold cross-validation
R2	Input Data Inconsistency	Implement frontend validation rules; provide tooltips and examples for each input field
R3	Privacy Concerns	Perform all processing locally; avoid external API calls; encrypt sensitive data
R4	UI Usability Issues	Conduct iterative user testing with feedback loops; use clean and accessible UI layouts

← END Summary

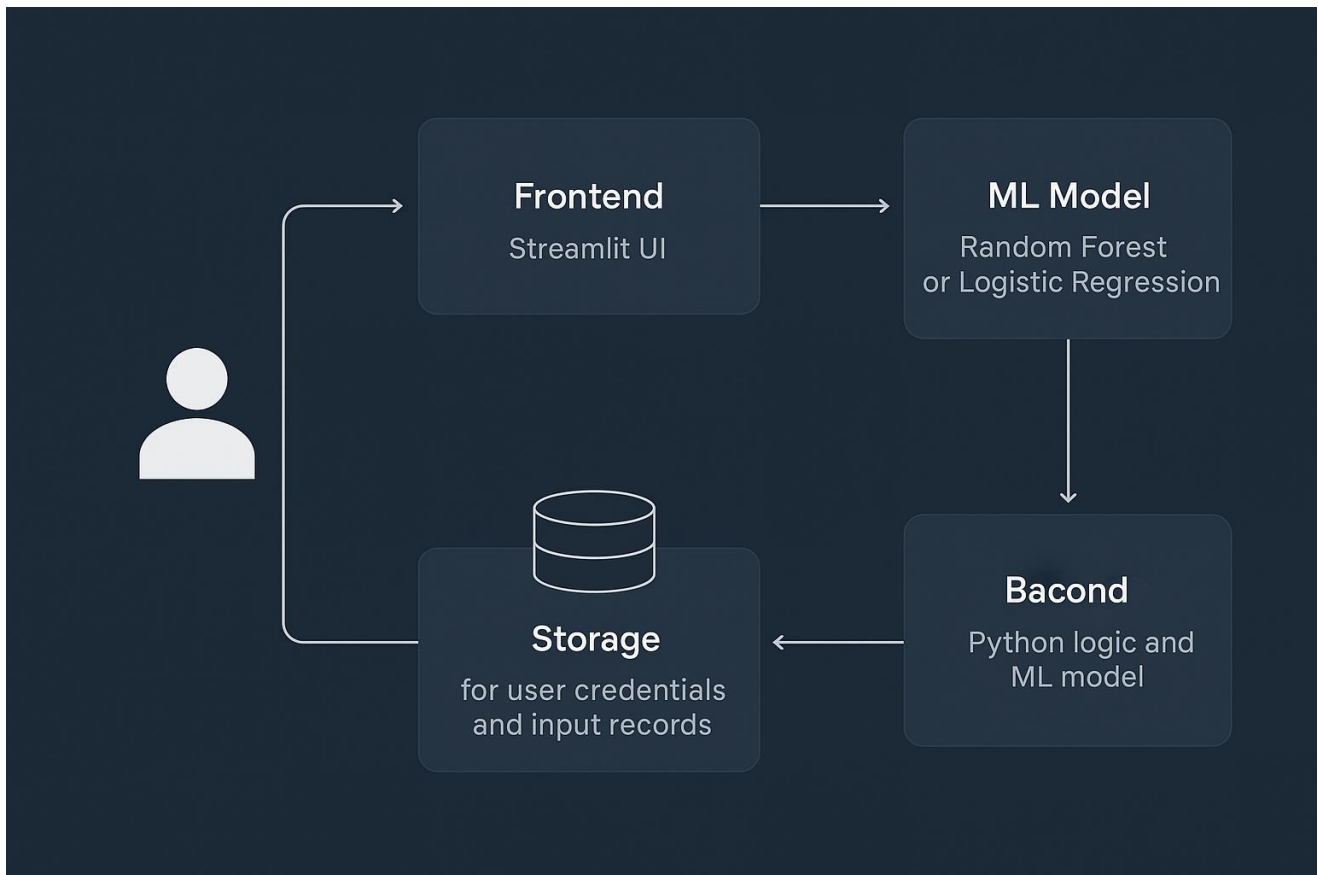
This WBS ensures every major functional and technical component of your heart disease prediction system is well-structured and executed efficiently. Risk management is also aligned with healthcare-specific concerns such as **data accuracy**, **user trust**, and **privacy**.

Experiment - 6

1. System Architecture

Components:

- **Frontend (Streamlit):**
 - **UI elements: forms, charts, buttons**
 - **Displays prediction results and export options**
- **Backend (Python):**
 - **Handles user registration/login**
 - **Validates input and runs the ML model**
 - **Manages report generation and database interactions**
- **Storage (SQLite):**
 - **Stores user data and prediction history**
- **ML Model:**
 - **Trained offline**
 - **Loaded at runtime to make predictions on submitted health data**



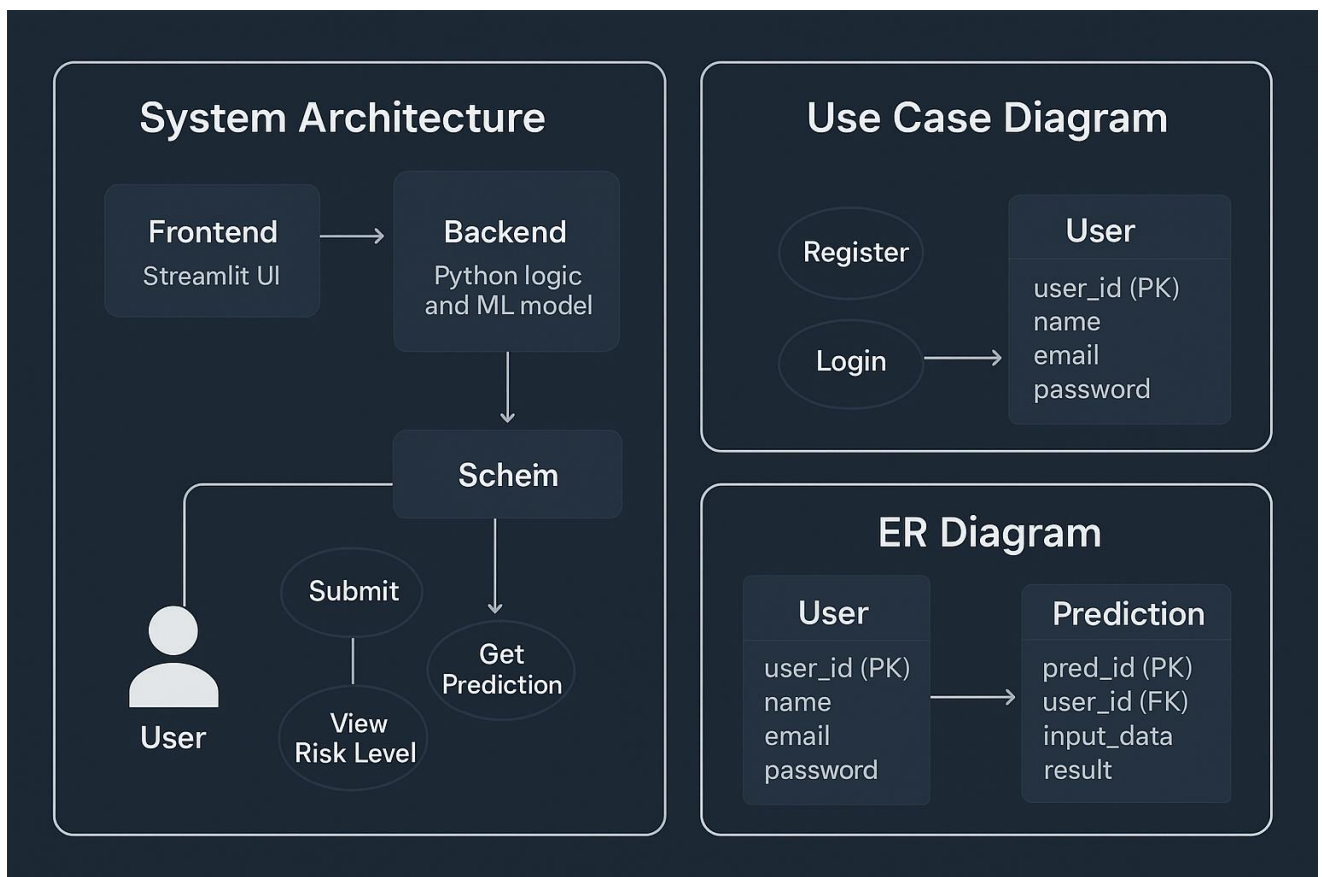
2. Use Case Diagram

Actors:

- **User:** Interacts with the system via Streamlit UI.

Use Cases:

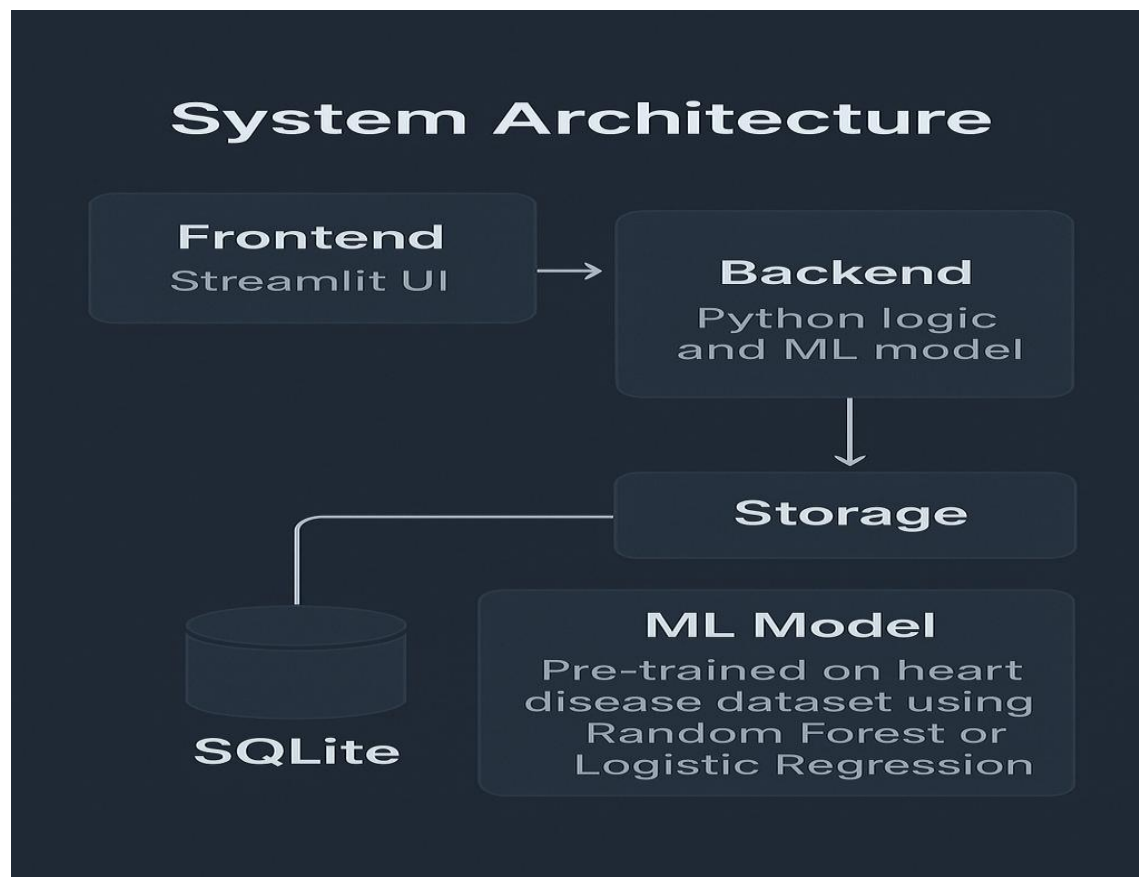
- **Register:** New user signs up with credentials.
- **Login:** Authenticates existing users.
- **Submit Data:** Users enter health parameters.
- **Get Prediction:** Backend ML model generates result.
- **View Risk Level:** Visualizes risk score and charts.
- **Export Report:** Generates downloadable PDF report.



3. Entity-Relationship (ER) Diagram

Entities & Attributes:

- **User**
 - **user_id** (Primary Key)
 - **name**
 - **email** (unique)
 - **password** (hashed)
- **Prediction**
 - **pred_id** (Primary Key)
 - **user_id** (Foreign Key referencing User)
 - **input_data**: Serialized string or JSON of input parameters
 - **result**: Prediction outcome (e.g., “At Risk”, “Healthy”) + probability
 - **timestamp**: DateTime of prediction



Experiment - 7

DFD LEVEL 0 (Context Diagram)

This is a high-level overview of the system. It shows how the user interacts with the system and the basic flow of data. For the heart disease prediction system:

- **User:** The user inputs data, which includes features like age, gender, cholesterol levels, blood pressure, etc.
- **System:** The system processes this data and uses the prediction model to predict the likelihood of heart disease.
- **Output:** The system displays the prediction (e.g., high risk, low risk, or no heart disease).

DFD Level 1 (Process Breakdown)

This level breaks down the system into its main modules. For the heart disease prediction system, you would have these modules:

1. **Authentication:** Users must authenticate to access the system (login or registration).
2. **Data Input:** Users provide personal health data that will be processed to predict the risk.
3. **Prediction Engine:** This module loads a trained model and makes predictions based on the input data.
4. **Result Display:** This module displays the prediction result, such as the likelihood of heart disease.
5. **Report Export:** The user may download or export the prediction results in the form of a report (e.g., PDF).

CLASS DIAGRAM

This diagram shows the structure of the system and its classes, along with their methods. You've mentioned three classes, and I'll elaborate on them for the heart disease prediction system.

1. User Class:

- **Attributes:** user_id, name, email, password, etc.
- **Methods:**
 - **register():** Registers a new user.
 - **login():** Authenticates the user.

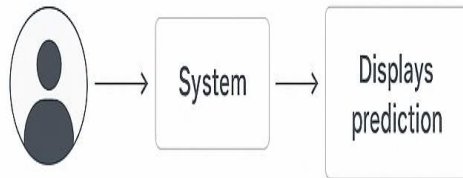
2. Predictor Class:

- **Attributes:** model, input_data, prediction_result.
- **Methods:**
 - **load_model():** Loads the pre-trained machine learning model for heart disease prediction.
 - **predict():** Uses the model to make a prediction based on user-provided input data.

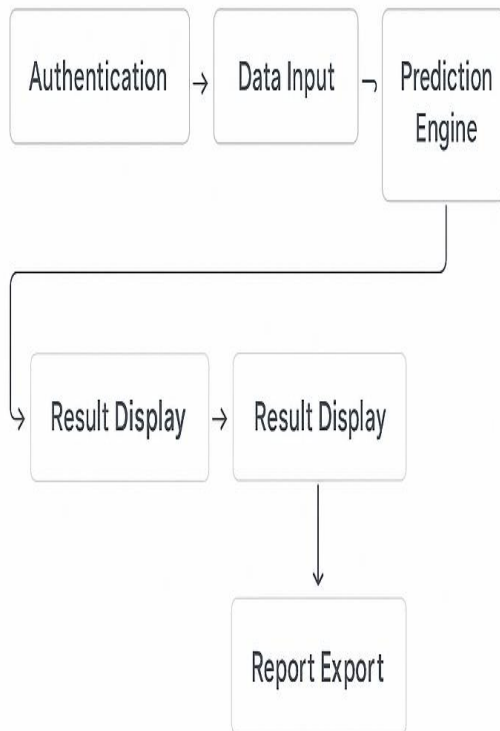
3. Report Class:

- **Attributes:** report_data, output_format.
- **Methods:**
 - **generate_pdf():** Exports the prediction results into a PDF format.
 - **visualize_result():** Visualizes prediction results with charts or graphs for easier understanding.

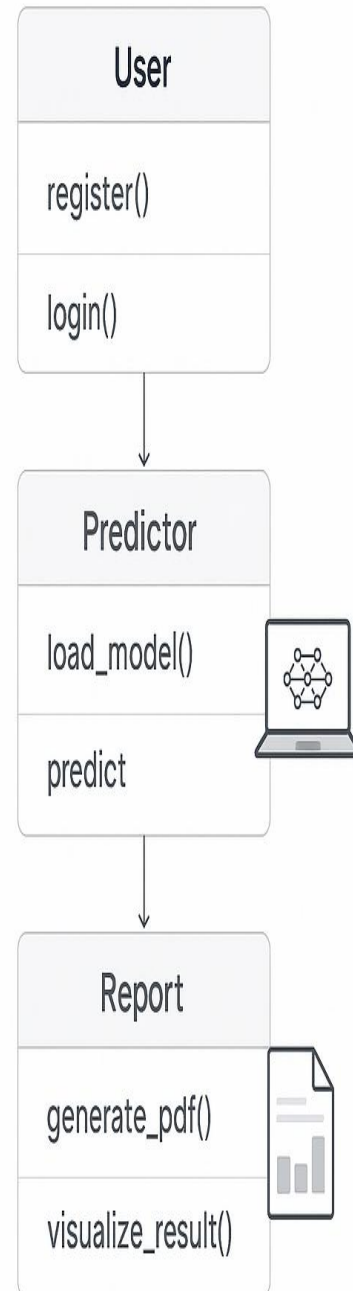
DFD Level 0 (Context Diagram)



DFD Level 1



Class Diagram



Experiment - 8

Interaction Diagram

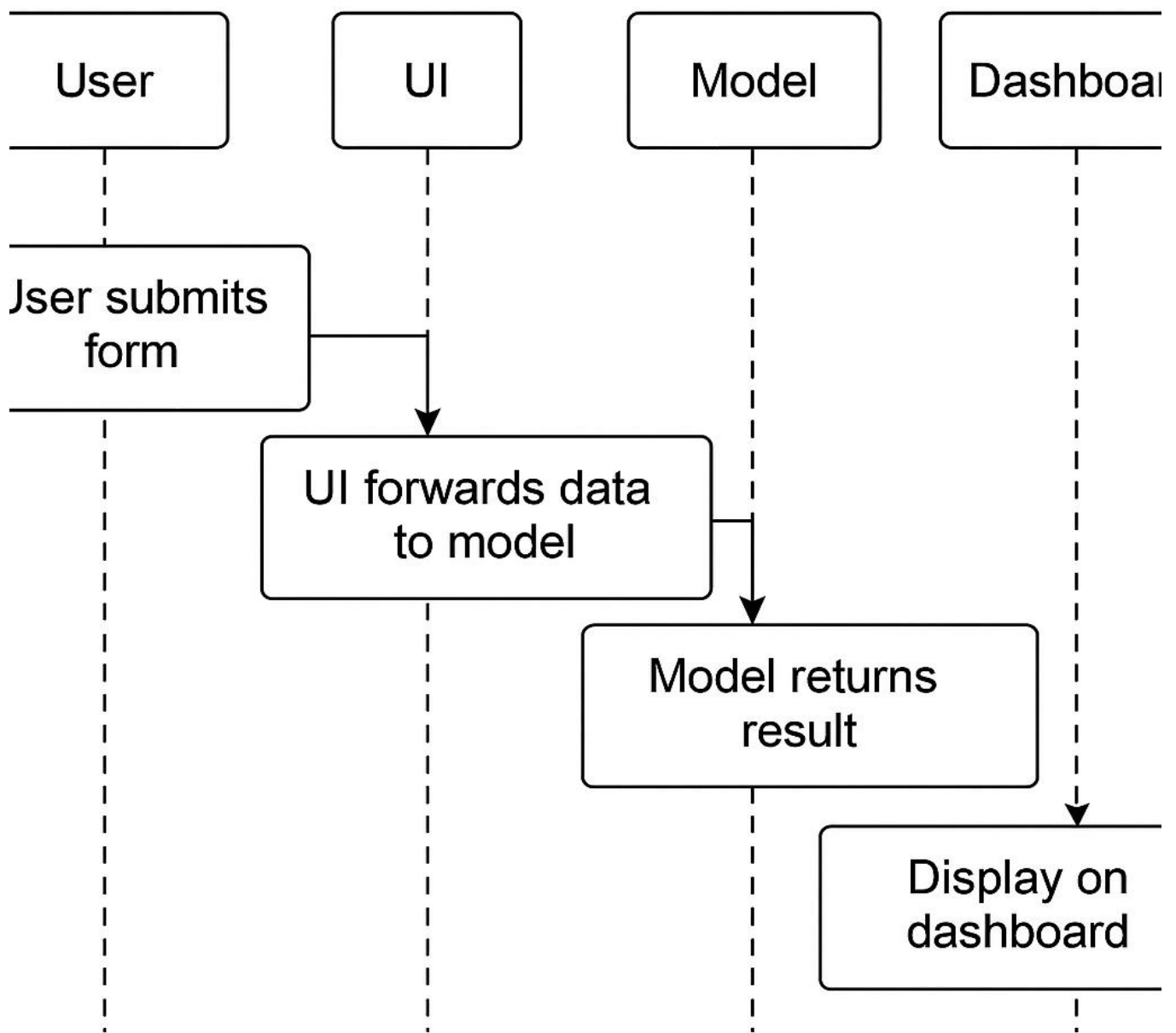
This shows how components interact over time when a user uses the heart disease prediction system.

Actors and Components:

- **User**
- **User Interface (UI)**
- **Prediction Controller**
- **ML Model**
- **Result Dashboard**

Sequence:

- 1. User fills and submits the form with medical details.**
- 2. UI captures the input and sends it to the Prediction Controller.**
- 3. Prediction Controller pre-processes the data.**
- 4. Sends the data to the ML Model.**
- 5. ML Model performs prediction and returns result.**
- 6. Controller forwards the result to the UI.**
- 7. UI displays result on the Result Dashboard.**
- 8. Optional: User can choose to download/export a report.**



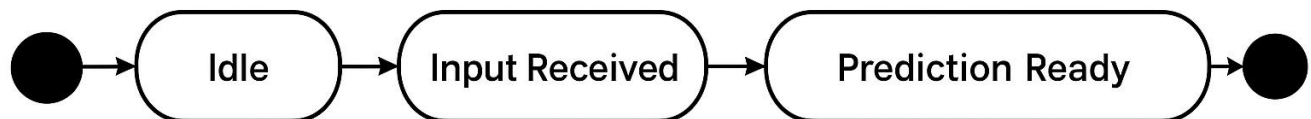
State Chart Diagram

This diagram shows the system states from start to end during prediction:

States and Transitions:

- Idle
(Initial state: system is awaiting user input)
→ on form submission →
- Input Received

- (User data captured)***
→ ***on data validation*** →
- **Processing**
(ML model runs prediction)
→ ***on prediction complete*** →
 - **Prediction Ready**
(Result is available and displayed to user)
→ ***on report export request*** →
 - **Report Exported**
(System exports report as PDF/CSV)
 - **Final State**
(Process ends or awaits new input)

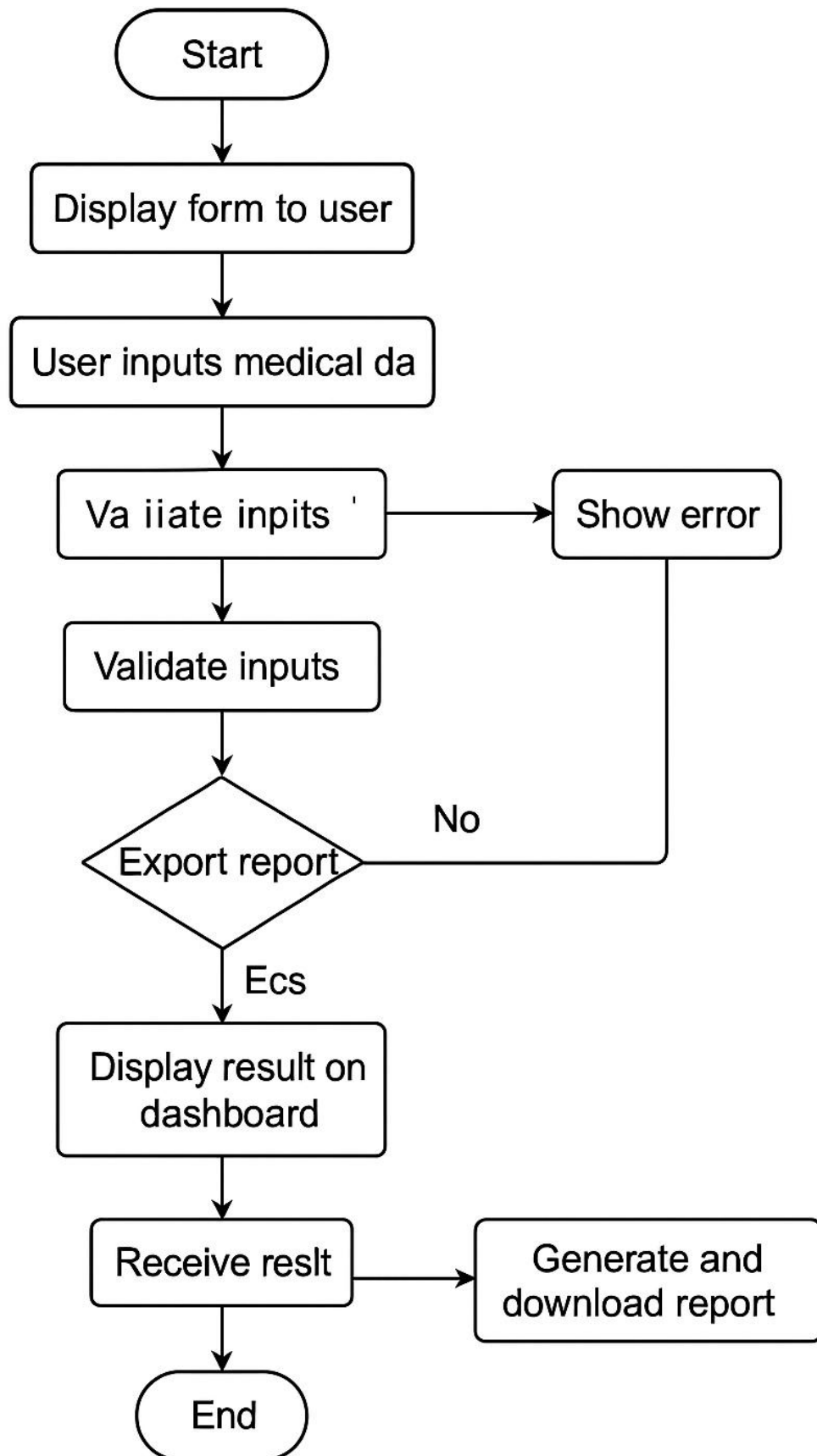


Activity Diagram

This diagram outlines the flow of operations in the system.

Flow:

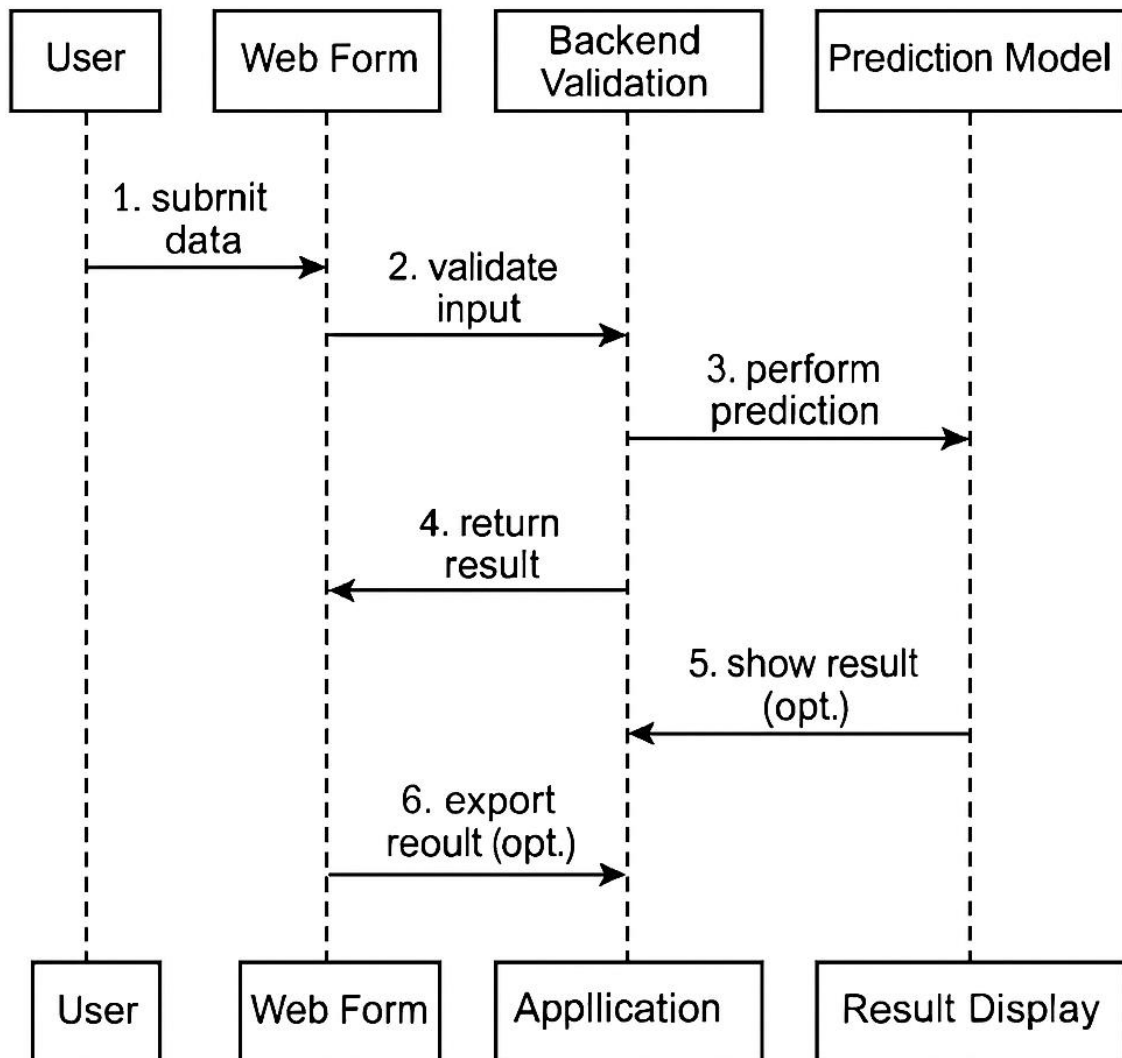
- Start
- → *Display form to user*
- → *User inputs medical data (e.g., age, BP, cholesterol)*
- → *Form submitted*
- → *Validate inputs*
 - If invalid → *Show error* → *Go back to input*
- → *Preprocess data*
- → *Send to ML model for prediction*
- → *Receive result*
- → *Display result on dashboard*
- → [Decision: Export report?]
 - Yes → *Generate and download report*
 - No → *End session or restart*
- End



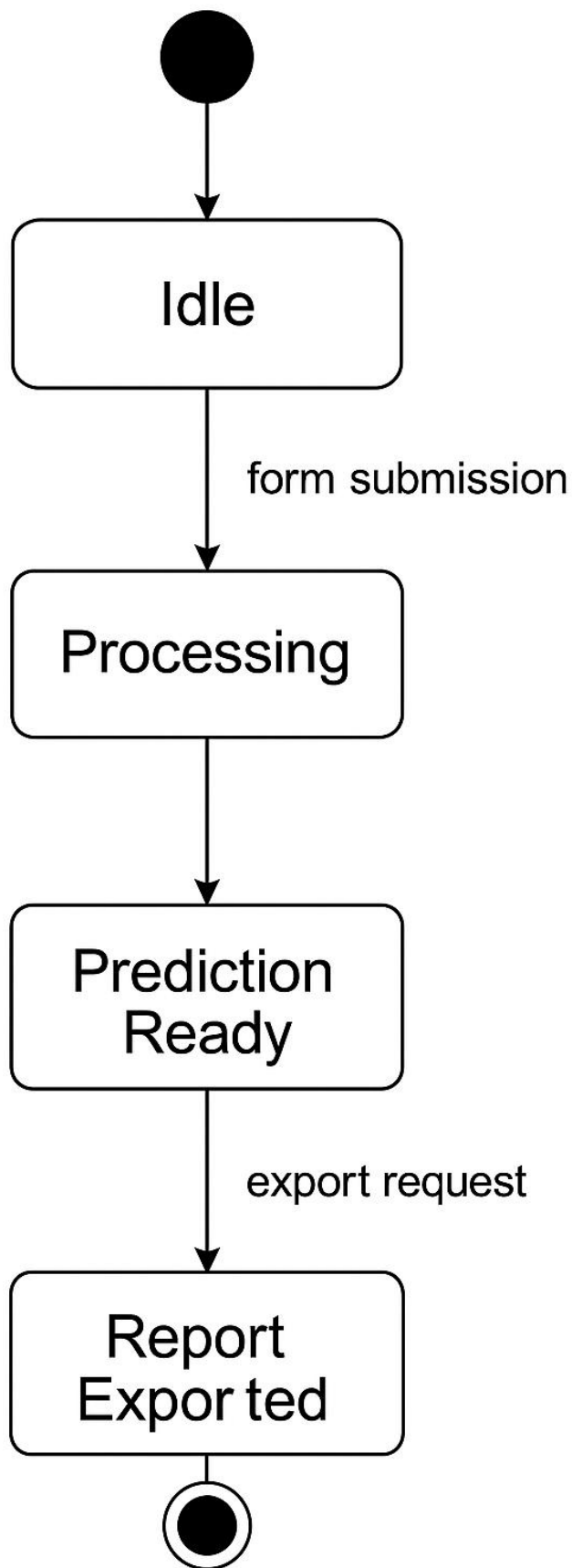
Activity Diagram for Heart Disease Prediction

Experiment No. 9

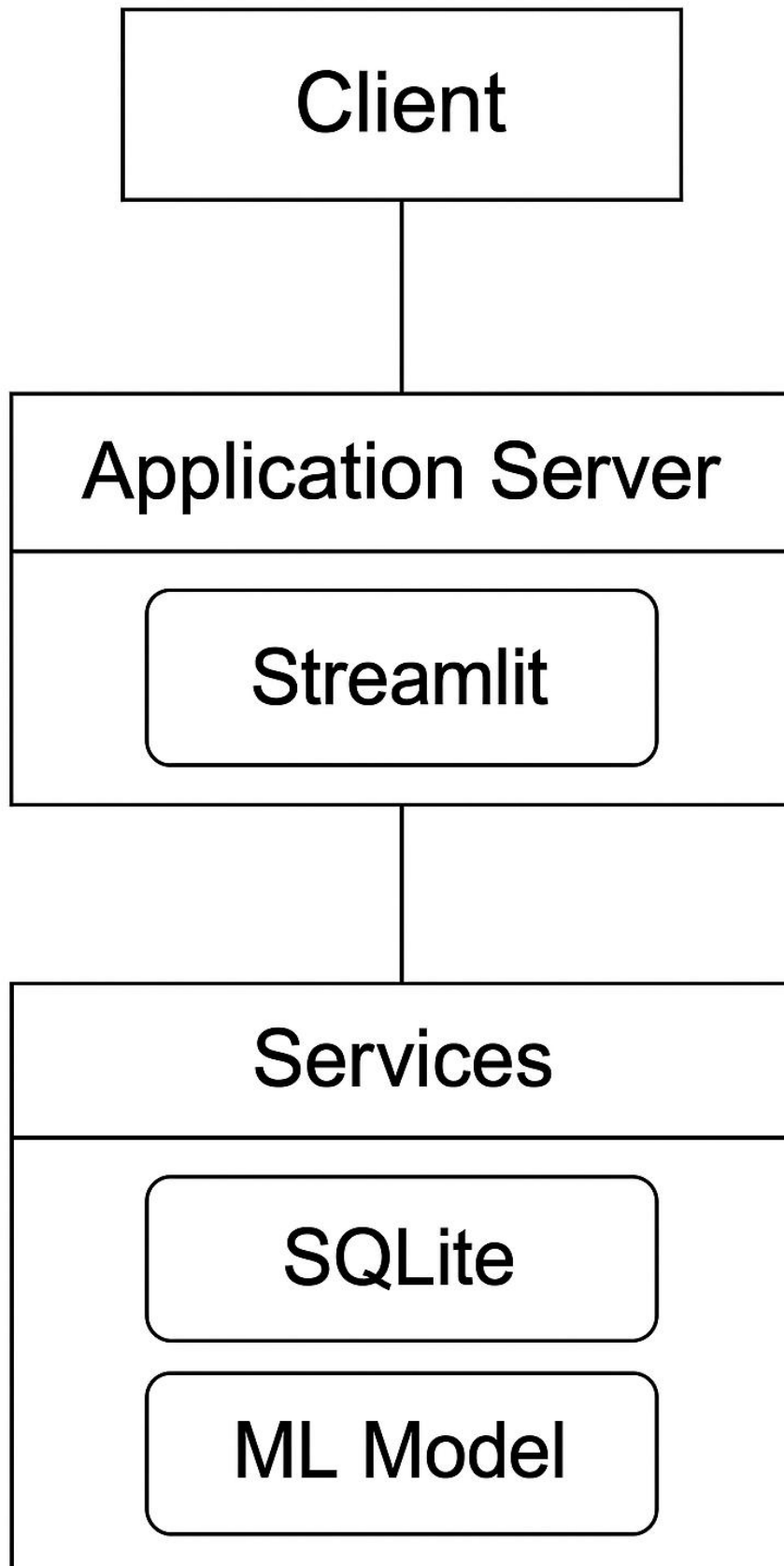
Sequence Diagram:



State Diagram:



Deployment diagram:



Experiment -10

1. Login Page:

- **Purpose:** Secure user access to the system.
- **Key Elements:**
 - **Username and Password Fields:** Clearly labeled, with an option for password visibility toggle.
 - **Login Button:** Prominently placed, with a hover effect to indicate interactivity.
 - **Forgot Password Link:** Easy-to-find for password recovery.
 - **Sign-Up Link:** If new users are allowed, a link to the registration page should be visible.
 - **Social Media Login (Optional):** For faster sign-ins (Google, Facebook, etc.).
- **Design Considerations:**
 - **Clean and Minimalistic:** Avoid clutter and focus on the login essentials.
 - **Branding:** Include logo and brand colors for consistency.
 - **Error Handling:** Display clear error messages if login credentials are wrong.
 - **Accessibility:** High contrast between text and background, proper font size, and alt-text for icons.

2. Data Entry Form:

- **Purpose:** Collect necessary data for processing or analysis.
- **Key Elements:**
 - **Input Fields:** Text fields for user data (e.g., name, age, other relevant details). Use placeholders to indicate the expected input.
 - **Dropdowns and Radio Buttons:** For selections such as gender or predefined options.

- **File Upload Option:** For any files required, with a drag-and-drop feature and clear file type/size limits.
- **Submit Button:** To send data for processing or prediction.
- **Design Considerations:**
 - **Field Grouping:** Group related fields together (e.g., personal information, file upload).
 - **Tooltips/Guidance:** Provide short descriptions or tooltips next to complex fields, explaining what data should be entered.
 - **Progress Indicators:** If the form requires a multi-step process, show a step indicator.
 - **Responsive Design:** Make sure the form adapts well to different screen sizes, especially mobile.
 - **Error Messages:** Display in-line error messages for incorrect input (e.g., wrong file format, missing information).

3. Prediction Dashboard:

- **Purpose:** Display results of predictions with relevant information.
- **Key Elements:**
 - **Summary of Predictions:** A clear and concise list or grid showing the results, e.g., "No Tumor," "Glioma," "Meningioma," etc.
 - **Visualization:** Graphs, pie charts, or bar charts showing data trends and classification breakdown.
 - **Real-Time Updates:** If predictions are being processed in real time, provide feedback (e.g., loading spinner, "prediction in progress" message).
 - **Navigation:** Allow easy access to other sections of the application (e.g., Data Entry, Settings).

- **Design Considerations:**
 - **Data Display:** Use contrasting colors for different tumor types to make distinctions clear.
 - **Responsive Layout:** Ensure that the layout adjusts well to different screen sizes (e.g., charts should be legible on mobile).
 - **Interactivity:** Allow users to interact with the data (e.g., hover over sections of a chart to get detailed statistics).
 - **Clear Labels:** Each chart/graph should have a descriptive title and axis labels for clarity.

4. Result with Visualization and Export Option:

- **Purpose:** Show detailed results of the predictions and allow users to export their data.
- **Key Elements:**
 - **Detailed Results Table:** A well-organized table displaying the predictions along with additional data like confidence scores.
 - **Visualization:** Advanced visualizations (e.g., heatmaps, segmented MRI images, etc.) to provide insights into predictions.
- **Design Considerations:**
 - **Export Button Visibility:** Ensure the export options are easy to find, with clear icons or labels (e.g., "Download Report").
 - **Visual Appeal:** Use smooth transitions for visual elements (charts, graphs) and ensure they are easy to understand.
 - **User-Friendly Data Export:** Make sure the exported files contain all necessary information in a well-structured format.
 - **Real-time Feedback:** Notify users once the export is complete and provide an option to view the exported file directly.

Design Elements:

Sidebar Navigation:

- **Purpose:** Provide easy access to different sections of the application.
- **Key Features:**
 - **Persistent Sidebar:** Should be visible throughout the user's navigation. Consider a collapsible sidebar for more screen space.
 - **Icons and Text:** Use clear icons with accompanying text to denote the sections (e.g., "Data Entry," "Results," "Export").
- **Purpose:** Keep users informed of ongoing processes.
- **Key Features:**
 - **Loading Indicators:** Use spinners or progress bars during data processing or prediction stages.
 - **Success/Failure Messages:** After a user action (e.g., form submission, file upload), show a notification about the success or failure.
- **Design Considerations:**
 - **Non-intrusive:** Feedback should not overwhelm the user. Use subtle animations or pop-up messages that are easy to dismiss.
 - **Color Coding:** Use green for success, red for errors, and yellow/orange for warnings.

Input Guidance/Tooltips:

- **Purpose:** Help users understand the expected inputs and provide extra information without cluttering the page.
- **Key Features:**
 - **Tooltips on Hover:** When users hover over a field or button, a small box appears providing additional instructions or examples (e.g., "Please upload a .jpg or .png file").

- **Design Considerations:**
 - **Keep It Simple:** Tooltips should be short, concise, and directly related to the field they refer to.
 - **Accessibility:** Make sure tooltips are screen reader-friendly and easy to read.

EXPERIMENT :11

Modules Implemented:

1. Authentication with Password Hashing:

Purpose: To secure user authentication by hashing the passwords. This ensures that passwords are not stored in plain text, enhancing the security of the system.

Steps Involved:

- Hash the password during registration.
- Compare the hashed password during login.

Libraries Used: bcrypt for hashing, flask for user authentication (if web-based).

2. Health Data Input Collection:

Purpose: Collect user-specific health data (age, cholesterol levels, blood pressure, etc.) for heart disease prediction.

Steps Involved:

- Create input forms for collecting health data from the user.
- This data will be used to make predictions about the likelihood of heart disease.

Libraries Used: flask (if using a web interface), pandas for data processing.

3. Prediction Using scikit-learn Model:

Purpose: Use a trained machine learning model to predict the likelihood of heart disease based on the user's health data.

Steps Involved:

- Collect health data from the user, preprocess it, and input it into the trained model.
- Return the prediction (e.g., "Low Risk," "Medium Risk," "High Risk").

Libraries Used: scikit-learn, joblib (to load the trained model).

4. Report Export Using FPDF:

Purpose: Generate a downloadable report (in PDF format) that includes the health data, prediction results, and a detailed analysis.

Libraries Used: FPDF for generating the PDF report.

5. Visual Feedback Using Matplotlib/Seaborn:

Purpose: Provide visual feedback such as charts or graphs to help users understand the risk and prediction results.

Libraries Used: matplotlib, seaborn.

EXPERIMENT :12

Test Cases:

TC01: Valid Login → Pass

Objective: Verify that the system allows valid users to log in successfully after entering correct credentials.

Test Steps:

- 1. Open the login page.**
- 2. Enter a valid username and the correct password (already stored in the database after being hashed).**
- 3. Click the "Login" button.**

Expected Outcome:

- The system authenticates the user by checking the hashed password against the stored password hash.**
- The user is redirected to the dashboard or the next page after successful login (e.g., health data input page).**

Pass Criteria:

- Login is successful, and the user is able to navigate to the next step in the application.**

Failure Scenario:

- The system fails to authenticate the user or throws an error message such as "Incorrect username or password."**

TC02: Form Validation with Missing Data → Error Prompt

Objective: Verify that the system correctly validates user inputs when some required fields are left empty or incorrectly filled.

Test Steps:

- 1. Navigate to the health data input form.**
- 2. Leave one or more required fields (e.g., age, cholesterol level, blood pressure) empty or filled**

incorrectly.

3. Click the "Submit" button.

Expected Outcome:

- The system detects the missing or invalid data and displays an error message or prompt informing the user that specific fields are required (e.g., "Please enter a valid age").

Pass Criteria:

- The system correctly identifies the missing or invalid input fields and prompts the user to correct them.

Failure Scenario:

- The system fails to display an error message or processes the incomplete form without validating the fields, leading to incorrect or incomplete data submission.

TC03: Model Output for Known Case → Accurate Result

Objective: Verify that the model provides the correct prediction based on a known test case (e.g., a case with predefined heart disease risk).

Test Steps:

1. Provide a known set of input data (e.g., age = 50, cholesterol = 240, blood pressure = 130) that should lead to a specific result (e.g., "High Risk").
2. Pass the data into the trained model for prediction.

Expected Outcome:

- The system uses the trained model to generate a prediction.
- The prediction should match the expected output for that known case (e.g., "High Risk").

Pass Criteria:

- The model returns the expected result for the known test case, matching the actual output of the model.

Failure Scenario:

- The system provides an incorrect prediction or fails to generate a result.

TC04: Report Export → PDF Generated Correctly

Objective: Verify that the system correctly generates and downloads a PDF report after the prediction is made, containing relevant user and prediction details.

Test Steps:

1. Enter valid health data (e.g., age, cholesterol, blood pressure) into the form.
2. After receiving the prediction (e.g., "High Risk"), click on the "Generate Report" or "Download Report" button.
3. The system generates a PDF document containing the user's health data, prediction results, and any relevant analysis.
4. Open the generated PDF file and verify its contents.

Expected Outcome:

- The system should generate a properly formatted PDF report.
- The report should include key sections like:
 - User's health data (e.g., age, cholesterol, blood pressure)
 - The heart disease prediction result (e.g., "Low Risk," "High Risk")
 - Any additional analysis or recommendations (if applicable).
- The PDF should be legible, with correct formatting and structure.

Pass Criteria:

- The PDF is correctly generated with all necessary information, and the file is downloadable.
- The report looks professional, with clear headings and a logical structure.

Experiment-13

Areas Tested:

1. UI Forms (Pass):

Objective: Test the user interface forms for collecting health data to ensure they are functional and user-friendly.

Test Focus:

- Ensure that all required fields (e.g., age, cholesterol level, blood pressure) are present.
- Verify that the input fields allow for valid data entry, and invalid or incomplete data is appropriately handled (e.g., missing or incorrectly formatted data should trigger error prompts).
- Check the responsiveness and usability of the forms across different screen sizes (if applicable).

Pass Criteria:

- All fields are clearly labeled with proper input validation (e.g., numeric fields for cholesterol).
- Error prompts or tooltips are shown when the user fails to provide valid data.
- The form submits correctly when all required data is entered.

Test Outcome:

- The UI forms work as intended. All fields are functional, and input validation is appropriately handled, making the form easy to use and navigate.

2. Model Prediction (Pass):

Objective: Test that the model produces correct and

accurate predictions based on the health data provided by the user.

Test Focus:

- Input different test cases (e.g., low, medium, and high-risk heart disease cases) into the system and ensure the predictions align with known or expected outcomes based on the model.
- Check that the model does not fail or produce errors for normal input ranges.

Pass Criteria:

- The model accurately predicts heart disease risk based on input data.
- The prediction result (e.g., "Low Risk," "Medium Risk," "High Risk") matches expectations for the given data.
- The system handles common input cases correctly without errors.

Test Outcome:

- The prediction function is working as expected. All predictions are accurate and in line with the trained model, and the system produces consistent results for known input cases.

3. Export Function (Pass):

Objective: Test that the system can successfully generate and export a report (e.g., PDF) after a prediction is made, with all relevant information included.

Test Focus:

- After a prediction is generated, ensure that the user can click a button or option to download or export the

report.

- Check that the report contains all necessary details, such as health data, the prediction result, and any additional analysis.
- Verify that the exported report is correctly formatted (e.g., PDF) and is legible.

Pass Criteria:

- The export function works, generating a PDF report without errors.
- The report contains all the expected information, formatted neatly and correctly.
- The PDF is downloadable, and the content is displayed as intended (no missing data or formatting issues).

Test Outcome:

- The export function is functioning correctly. Reports are generated accurately, and all required data (health info, prediction) are included in the exported file without formatting issues.

4. Edge Case Handling (Pass):

Objective: Ensure the system can handle edge cases, such as unusual or extreme input values, without crashing or producing incorrect results.

Test Focus:

- Test the system with edge case inputs (e.g., extremely high or low values for age, cholesterol, or blood pressure) to see how the model and the application handle them.
- Test inputs that are on the boundaries of valid ranges

(e.g., cholesterol exactly at the threshold for "high risk" levels).

- Ensure that invalid inputs (e.g., negative numbers or strings where numbers are expected) are correctly handled by the system.**

Pass Criteria:

- The system correctly handles edge cases without crashing or producing errors.**
- Invalid inputs trigger appropriate error messages, and valid edge cases result in reasonable predictions.**
- No unexpected behavior or system failures occur for edge cases.**

Test Outcome:

- The system handles edge cases effectively. For instance, extreme health values do not cause system crashes, and the model continues to generate accurate predictions or error messages when necessary.**

Experiment 14:

User Manual Instructions:

This section explains how the user should interact with your heart disease prediction system, from registration to generating and saving reports.

1. Register/Login to the System:

- **Step 1: Open the application and navigate to the login page.**
- **Step 2: If you're a new user, click "Register" and create an account by providing a username and a password. Ensure the password is strong and meets the required criteria (e.g., minimum length).**
- **Step 3: If you're already registered, enter your username and password in the login form, then click "Login."**
- **Step 4: Upon successful login, you'll be directed to the health data input page where you can enter your details.**

Note: Passwords are hashed for security, ensuring that your credentials are safely stored.

2. Enter Required Health Data:

- **Step 1: After logging in, you will be directed to a health data input form.**
- **Step 2: Complete the form by entering the following information:**
 - **Age: Your current age.**
 - **Cholesterol Level: Your cholesterol reading in mg/dL.**

- **Blood Pressure: Your systolic/diastolic blood pressure reading.**
- **Step 3: If any field is missing or incorrect, an error message will prompt you to correct it before proceeding.**
- **Step 4: Once all fields are filled out correctly, click the "Predict" button to submit the data for analysis.**

3. Click "Predict" to View Risk Status:

- **Step 1: After clicking "Predict," the system will use a machine learning model to calculate the likelihood of heart disease based on the inputted data.**
- **Step 2: The system will display a risk status, such as "Low Risk," "Medium Risk," or "High Risk."**
- **Step 3: If you wish to review the details or prediction further, you can explore any provided visual feedback or charts that accompany the result.**

4. Click "Export" to Save the Result as PDF:

- **Step 1: After viewing the prediction result, you can choose to generate a detailed report.**
- **Step 2: Click the "Export" button to download a PDF version of the report.**
- **Step 3: The report will contain your health data, the prediction result, and any additional analysis or recommendations (if included).**
- **Step 4: The PDF report will be available for download immediately.**

EXPERIMENT:15

Experiment 15: Final Project Summary

The Heart Disease Prediction System is a comprehensive, real-world application that leverages data science techniques to assist with the early diagnosis of heart disease. This project is a prime example of how technology and healthcare can combine to provide accessible, data-driven insights for patients and healthcare professionals. Below is an overview of how the system was developed and its significance.

Project Overview:

The Heart Disease Prediction System serves as a tool for assessing the risk of heart disease based on user input data, such as age, cholesterol level, and blood pressure. The system uses a machine learning model to predict whether a user is at high, medium, or low risk of developing heart disease. Additionally, it generates detailed reports, which can be exported in PDF format for further analysis or sharing with healthcare providers.

Key Phases of the Software Development Life Cycle (SDLC):

1. Requirement Analysis:

- Objective: Understand the problem space and define the user requirements.**
- Key requirements included the ability for users to input health data, receive a prediction**

regarding their heart disease risk, and generate a downloadable report containing the results.

- The system was designed to be easy to use, ensuring accessibility even for users with limited technical knowledge.

2. Design:

- **User Interface (UI):** The design focused on simplicity and ease of use. A user-friendly interface was created using Streamlit, allowing users to enter their health data and receive predictions in a seamless manner.
- **Model Architecture:** A machine learning model was developed using scikit-learn. It was trained on a heart disease dataset and used to predict the risk of heart disease based on user inputs.
- **Report Generation:** The FPDF library was integrated to generate customizable PDF reports that include the user's health data, the model's prediction, and any further insights.

3. Development:

- **Backend Development:** The core logic for handling user input, generating predictions, and exporting reports was implemented in Python using libraries such as scikit-learn for machine learning and FPDF for PDF export.
- **Frontend Development:** The UI was built using Streamlit, providing an interactive platform where users could enter their data and view

their predictions.

4. Testing:

- **Extensive manual testing was carried out to ensure that all components of the system functioned as intended. This included testing for UI functionality, model accuracy, report generation, and edge case handling.**
- **A variety of test cases were executed to confirm that the system could handle valid and invalid inputs, produce accurate predictions, and generate reports correctly.**

5. Delivery:

- **After successful testing and validation, the project was delivered as a fully functional web-based application. The system was deployed using Streamlit, making it accessible to users through a web interface. The project was designed to be lightweight and efficient, ensuring it can be used even in low-resource settings.**

